

# JournaBuddy – Complete System Design & Implementation Plan

**Author:** Abdullah Al Mamun (ORCID: 0009-0006-7473-0024)

**Project:** JournaBuddy – Research Paper Intelligence Platform

**Status:** Module 1 MVP built. Transitioning to production-grade architecture.

**Goal:** Build a full hybrid system with modern UI, data security, error handling, mathematical provenance, and horizontally scalable, **100% free and open-source** software.

---

## 1. Vision & Core Philosophy

JournaBuddy helps authors prepare manuscripts by combining three analysis layers: - **Symbolic / rule-based** – precise, explainable checks (Module 1). - **Statistical NLP** – continuous, benchmarkable metrics (Module 4). - **Semantic / embedding + knowledge graph** – meaning and relationships (Modules 2, 4).

The system answers four core questions: | Question | Module | |———→|———|  
| *Is my paper ready to submit?* | Module 1: Manuscript Readiness Checker | |  
| *Which journals fit my paper?* | Module 2: Journal Discovery & Matching | |  
| *Are those journals trustworthy?* | Module 3: Journal Trust Scorer | |  
| *How does my paper compare to its field?* | Module 4: BI Dashboard & Benchmarking |

**Core principles:** radical transparency, hybrid intelligence, user-centric design, security & privacy by design, zero-cost infrastructure (no API keys, no usage limits, no vendor lock-in).

---

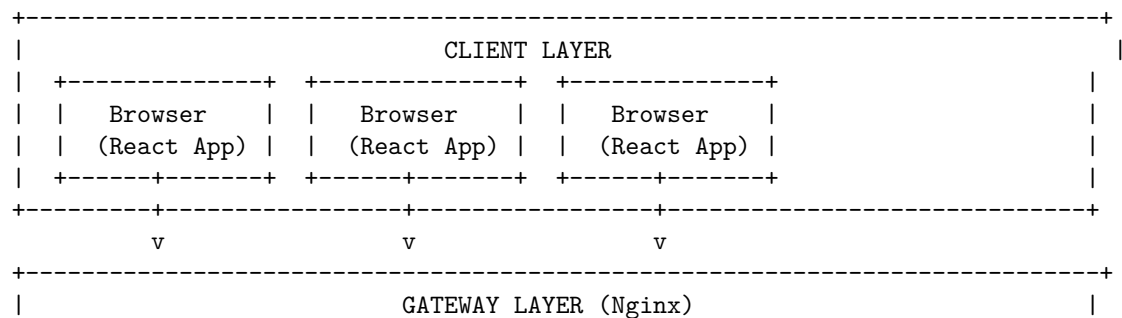
## 2. Technology Stack

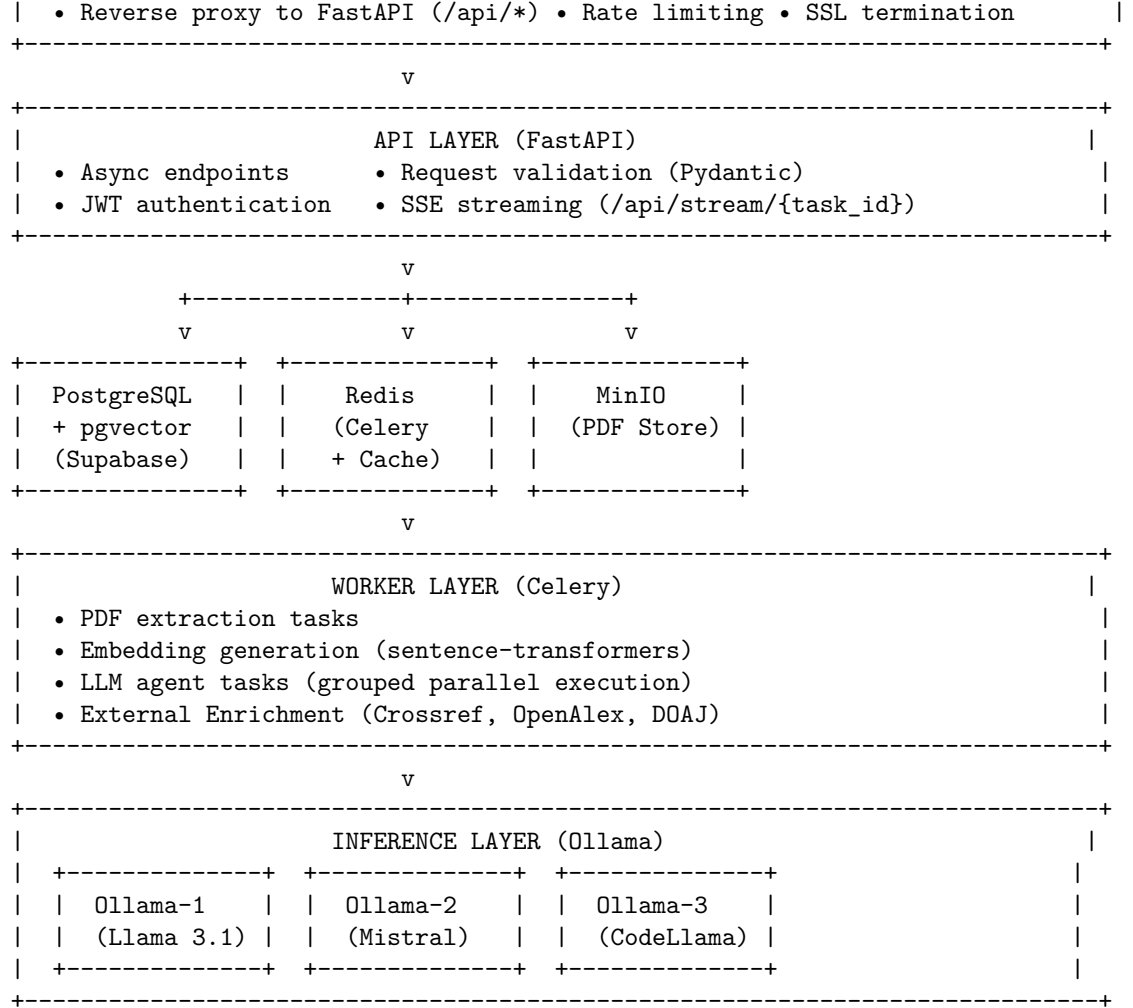
| Layer           | Technology                   | License    | Purpose                           |
|-----------------|------------------------------|------------|-----------------------------------|
| <b>Frontend</b> | React 19 + TypeScript + Vite | MIT        | UI framework                      |
| <b>Styling</b>  | TailwindCSS 4                | MIT        | Utility-first CSS & Glassmorphism |
| <b>Charts</b>   | Apache ECharts & vis-network | Apache/MIT | Interactive dashboards & graphs   |
| <b>State</b>    | Zustand                      | MIT        | Global state management           |
| <b>Query</b>    | TanStack Query               | MIT        | Server state & caching            |

| Layer                | Technology                      | License    | Purpose                                       |
|----------------------|---------------------------------|------------|-----------------------------------------------|
| <b>Backend</b>       | FastAPI (Python)                | MIT        | Async API server                              |
| <b>ORM</b>           | SQLAlchemy 2 + Alembic          | MIT        | Database models & migrations                  |
| <b>Task Queue</b>    | Celery                          | BSD        | Distributed task processing                   |
| <b>Broker/Cache</b>  | Redis                           | BSD        | Celery broker, response caching & rate limits |
| <b>Database</b>      | PostgreSQL 16 (via Supabase)    | PostgreSQL | Relational data                               |
| <b>Vectors</b>       | pgvector (PostgreSQL extension) | MIT        | Embedding storage & search                    |
| <b>Object Store</b>  | MinIO                           | AGPLv3     | PDF storage (S3-compatible)                   |
| <b>LLM Inference</b> | Ollama                          | MIT        | Local LLM serving (Llama 3.1 / Mistral)       |
| <b>Embeddings</b>    | sentence-transformers           | Apache 2.0 | Text embeddings (all-MiniLM-L6-v2)            |
| <b>PDF Parsing</b>   | pdfplumber + PyMuPDF            | MIT        | Text & layout extraction                      |
| <b>Gateway</b>       | Nginx                           | BSD        | Reverse proxy, static files, rate limiting    |
| <b>Monitoring</b>    | Prometheus + Grafana + Jaeger   | Apache 2.0 | Metrics, tracing, dashboards                  |

### 3. High-Level Architecture & Data Flow

#### 3.1 System Architecture





### 3.2 End-to-End Hybrid Data Flow

1. **Upload:** PDF/DOCX uploaded → extracted via `pdfplumber` / stored in MinIO.
2. **Symbolic Phase:** Run structure, acronyms, citation-reference, and grammar (LanguageTool) checks.
3. **Statistical NLP:** Readability, passive voice, lexical diversity, jargon extraction (textstat, spaCy).
4. **Semantic Phase:** Embed abstract, extract keywords (KeyBERT), compute topic similarity.
5. **Agent Orchestration (LLM):** Parallel Celery agents execute specific analysis tasks using Ollama.
6. **Async Enrichment:** Resolve references via Crossref, fetch citation data

from OpenAlex, build citation graph.

7. **Benchmarking:** Compare manuscript against field benchmarks (precomputed nightly).
8. **Convergence:** Aggregate all results into JSON payload with provenance IDs and stream to frontend via SSE.

---

## 4. Component Deep Dive

### 4.1 FastAPI & Server-Sent Events (SSE)

File uploads are I/O bound. FastAPI's `async/await` allows a single process to handle hundreds of concurrent uploads without blocking. We replace polling with SSE for real-time frontend updates:

```
@router.get("/stream/{task_id}")
async def stream_task(task_id: str):
    async def event_stream():
        pubsub = redis.pubsub()
        pubsub.subscribe(f"task:{task_id}")
        async for message in pubsub.listen():
            if message["type"] == "message":
                yield f"data: {message['data']}\n\n"
    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

### 4.2 Agent Orchestration (Celery + Redis)

11 separate LLM calls cause high latency. We optimize this using a **Celery Chord** (parallel groups + sequential callback): - **Phase 1 (I/O):** Extract PDF, chunk, and embed. - **Phase 2 (Parallel LLM - 6 concurrent calls):** - *Group A:* Document Intelligence (title, authors, keywords) - *Group B:* Language & Compliance (grammar, citations) - *Group C:* Originality Analysis (plagiarism risk, novelty) - *Group D:* Research Rigor (methodology score) - *Group E:* Submission Readiness (journal fit) - *Group F:* Peer Review Simulation (3 personas) - **Phase 3 (Dependent Chain):** Truth Checker → Quality Checker → Convergence.

### 4.3 Caching & Cost Optimization

- **Content-Addressable Cache:** Hash the extracted text. If the same paper is uploaded, return cached results instantly (100% savings).
- **Embedding Cache:** Embeddings are deterministic. Cache them in Redis by chunk content hash (30% savings).
- **LLM Response Cache:** Cache deterministic prompts (like metadata extraction) in Redis (15% savings).

## 5. Database Schema (PostgreSQL + pgvector)

Using pgvector unifies relational and vector data, reducing operational complexity.

```
CREATE EXTENSION IF NOT EXISTS vector;

-- Tasks & Orchestration
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  status VARCHAR(50) DEFAULT 'pending',
  dashboard_payload JSONB,
  progress_percent INTEGER DEFAULT 0
);

-- Semantic Embeddings (pgvector)
CREATE TABLE document_chunks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID REFERENCES tasks(id) ON DELETE CASCADE,
  chunk_index INTEGER,
  text_content TEXT,
  embedding vector(384) -- all-MiniLM-L6-v2 produces 384-dim vectors
);
CREATE INDEX ON document_chunks USING ivfflat (embedding vector_cosine_ops);

-- Provenance Log (Radical Transparency)
CREATE TABLE provenance_log (
  id SERIAL PRIMARY KEY,
  task_id UUID REFERENCES tasks(id),
  metric_name TEXT,
  metric_value JSONB,
  formula TEXT,
  data_sources TEXT[],
  confidence_level TEXT,
  raw_data_snapshot JSONB
);

-- Journals & Metrics
CREATE TABLE journals (
  id SERIAL PRIMARY KEY,
  issn TEXT UNIQUE,
  title TEXT,
  scope_embedding VECTOR(384)
);
```

---

## 6. Provenance & Truthfulness Layer

Every metric displayed on the frontend is backed by a verifiable source.

**Implementation:** - `ProvenanceEngine` logs every calculation into the `provenance_log` table. - Frontend fetches provenance via `/api/provenance/{id}` and displays expandable tooltips on hover.

| Metric      | Formula / Method             | Source              | Confidence |
|-------------|------------------------------|---------------------|------------|
| Readability | Flesch-Kincaid               | textstat            | High       |
| Journal fit | 0.6 semantic + 0.4 citedness | pgvector + OpenAlex | Medium     |
| Trust score | sum(signals)/count(signals)  | DOAJ, COPE, OASPA   | High       |
| LLM output  | Structured JSON Schema       | Ollama (Llama 3.1)  | Varies     |

## 7. Frontend & UI Design System

**UI/UX Paradigm:** Modern Glassmorphism - **Colors:** Primary: #2b4c3f (Sage Green), Accent: #38bdf8, Glass: `rgba(255,255,255,0.75)` with blur. - **Widgets:** Radar charts (Quality), Gauge charts (Readiness), Citation network (vis-network), Heatmaps (Plagiarism). - **State Management:** TanStack Query handles server state (auto-refetches on focus); Zustand handles UI toggles and preferences.

## 8. Security, Privacy & Error Handling

- **Data Privacy:** On-premise processing. Vectors and LLM inputs never touch third-party APIs. MinIO buckets are private. Files deleted after 2h.
- **Resilience:** Celery auto-requeues failed tasks. LLM calls wrapped in `tenacity` for exponential backoff retries.
- **Input Sanitization:** 50MB max file size. PDF sandboxing (extracted in network-isolated containers).
- **Observability:** Prometheus tracks metrics (tokens consumed, task latency). Loki aggregates JSON logs. Jaeger provides distributed tracing (e.g., [Upload] → [Extract] → [Embed] → [Group A]).

## 9. Docker Compose Deployment & Hardware

```
# Abridged docker-compose.yml
services:
  api:
    build: ./backend
    command: uvicorn app.main:app --workers 4
  worker-llm:
    build: ./backend
    command: celery -A app.worker.celery_app worker -Q llm_bound -c 3
  postgres:
    image: ankane/pgvector:latest
  minio:
    image: minio/minio
  ollama-1:
    image: ollama/ollama
  prometheus:
    image: prom/prometheus
```

**Hardware Requirements:** - **Development (1 user):** 4 cores, 16 GB RAM (CPU inference runs at ~5-10 tokens/sec). - **Small Team (5-10 users):** 8 cores, 32 GB RAM, RTX 3060 12GB. - **Production (50+ users):** 16+ cores, 64 GB RAM, RTX 4090 24GB.

---

## 10. Unified Implementation Roadmap

### Phase 1: Foundation & Data Infrastructure (Weeks 1-2)

- Docker Compose setup (FastAPI, Postgres+pgvector, MinIO, Redis).
- Rotate API keys, setup `.env`.
- Build basic PDF extraction pipeline with upload endpoint.
- Scaffold React 19 + Vite frontend.

### Phase 2: Intelligence Pipeline & Celery (Weeks 3-4)

- Migrate to Celery task queue; implement 6 parallel agent groups.
- Integrate Ollama (Llama 3.1:8b) for local LLM inference.
- Build semantic chunking & sentence-transformers embeddings.
- Implement the `ProvenanceEngine` and `provenance_log` logic.

### Phase 3: External Enrichments & Semantic Layers (Weeks 5-6)

- Integrate `textstat`, `spaCy`, and `KeyBERT` (Module 4).
- Setup Async integrations (Crossref, OpenAlex, DOAJ).
- Implement Journal Trust Scorer (Module 3) and Discovery (Module 2).

#### **Phase 4: Real-time UI & Dashboard (Week 7)**

- Build SSE streaming endpoint.
- Develop interactive Glassmorphism UI (ECharts, PDF side-by-side view).
- Implement provenance tooltips.

#### **Phase 5: Production Hardening (Week 8)**

- Content-addressable caching (Redis).
- Prometheus + Grafana + Jaeger observability setup.
- JWT Authentication, rate limiting (`slowapi`), and security sweeps.
- GitHub Actions for nightly OpenAlex benchmarking.

**Conclusion:** This hybrid plan successfully combines rigorous symbolic logic, provenance tracking, and deep system architecture into a production-ready, fully open-source environment.